Rethinking Agility In The Age Of Agents

David-Olivier Saban, with the help of Claude Sonnet 4.6 and GPT-5.5

Rethinking Agility In The Age Of Agents

Condensed Management Version

Author: David-Olivier Saban, with the help of Claude Sonnet 4.6 and GPT-5.5
Target audience: Engineering Managers, Heads of Engineering, Product Leaders, transformation leaders.
Objective: explain how to start an agentic transformation without reducing it to the purchase of AI tools.

Available versions:

Document	Title	Usage
Management version	**Rethinking Agility In The Age Of Agents**	Condensed version for management, Heads of Engineering and Product Leaders.
Long version	**The Death Of Manual Code: Rethinking The SDLC[1] In The Age Of Agents**	Detailed version for Engineering Managers and Tech Leads.

[1] **SDLC (Software Development Life Cycle)**: The full lifecycle of software development: idea, requirement, user story, design, implementation, validation, release and production.

Executive Summary

AI does not only transform code writing. It transforms the delivery system.

This document is the condensed management version. The long version, **The Death Of Manual Code: Rethinking The SDLC[2] In The Age Of Agents**, contains the technical details and complete examples.

The message for management is direct: buying licenses is not enough. You need to build an agentic setup[3]: versioned documentation, short instructions, deterministic tools[4], properly sized CI/CD[5], PR[6] environments, code owners, guardrails and validation rules.

Value appears when the whole team uses the same flow: PM[7], CSM[8], QA[9], designers, developers and agents[10]. The person carrying the need validates the functional outcome; the code owner validates the technical implementation.

---

[2]**SDLC (Software Development Life Cycle)**: The full lifecycle of software development: idea, requirement, user story, design, implementation, validation, release and production.

[3]**Agent**: Specialized configuration combining a role, behavior rules, forbidden actions and the ability to use tools. A developer agent does not behave like a QA or PM agent.

[4]**Tool**: Automated deterministic action: create a worktree, launch the local application, run validation. What must always execute in the same way should not depend on a prompt.

[5]**CI/CD**: Continuous Integration / Continuous Deployment. Automated pipeline that builds, tests and deploys changes.

[6]**Pull Request (PR)**: Change proposal submitted for validation in a Git repository. Every change, whether it comes from a human or an agent, goes through a PR before being integrated.

[7]**PM (Product Manager)**: Product role responsible for the problem, prioritization, expected impact and alignment with business objectives.

[8]**CSM (Customer Success Manager)**: Customer-facing role that surfaces field irritants, qualifies user needs and helps validate functional value.

[9]**QA (Quality Assurance)**: Quality function that checks product behavior, regressions, acceptance criteria and release risks.

[10]**Skill**: Reusable knowledge that can be used by several agents: writing a user story, auditing a PR, conducting a security review, maintaining documentation.

Promise And Observed Results On My Test Project

The promise is not only to code faster. The promise is to reduce delivery friction end to end.

On my test project, still in prototyping, the signals are already concrete:

Dimension	Observed result
Delivery	A much smaller team delivers as much as a previous team of about twelve people.
Coordination	Fewer handoffs, fewer meetings, faster decisions.
CI/CD	Capacity increased from about 3 simultaneous builds to about 20 simultaneous builds, without an equivalent cost increase thanks to preemptible infrastructure.
Monorepo	CI/CD, infrastructure, documentation, CMS, UI and E2E tests can live together, avoiding up to six repositories, pipelines and agentic structures.
Small changes	A CMS change such as a color picker was written, coded, validated and merged in less than one hour, with about 30 minutes of cumulative human time.
Quality of life	PMs/CSMs can carry low-risk irritants with an agent, if the code owner validates the PR.
PR quality	Well-prepared and well-contextualized PRs went from around ten review comments to often 0-2 comments.

These numbers are not a universal promise. They show what becomes possible when the technical and organizational ground is ready.

Thesis For Managers

The real topic is not AI. The real topic is how work is organized around AI.

Gains appear when three conditions are met:

1. **Context engineering**[11] **is in place**: documentation, specifications, decisions, workflows and constraints are versioned and accessible in the repository, to agents as well as humans.
2. **The setup is shared**: developers, PMs, CSMs, QA and designers use the same references, the same tools, the same guardrails and a shared flow from user story to production.
3. **Validation is clear**: the person carrying the need validates the functional outcome; the code owner validates the technical implementation.

Without these three conditions, AI adds noise. With them, it increases speed, autonomy and flow quality.

---

[11] **Context engineering**: Design and maintenance of the information required for a human or an agent to work correctly: documentation, specifications, decisions, constraints, workflows.

What Changes In Agility

Agents change the pace. Some tasks move from several days to less than an hour; product, architecture and prioritization decisions remain human. This gap creates tension: ceremonies designed to synchronize slow work become too heavy for small changes, but remain useful for arbitration, prioritization and risk sharing.

In my test project, ceremonies therefore did not disappear. They became less focused on reporting and more focused on flow regulation. The standup is used to decide whether a bug or irritant should be handled by a developer, a CSM with Copilot, an autonomous agent, or postponed. Refinement is less about writing the solution and more about clarifying impact, risk and acceptance criteria. The retrospective is used to understand why an agent failed: missing context, missing tool, unstable test, ambiguous instruction.

The following table summarizes this adaptation:

Ritual	Management decision
Standup	Become a regulation point: who takes what, human or agent, with what risk.
Refinement	Focus on impact, risk, debt and reversibility.
Sprint	Keep what helps, but absorb small changes in a continuous Scrumban[12] flow.
Demo	Show more often, ideally on a PR environment.
Retro	Audit agent failures: missing context, missing tool, unstable test, ambiguous instruction.

[12] **Scrumban**: Hybrid mode: Scrum structure (standup, retro, product vision) combined with a continuous Kanban flow for small changes.

The Agentic Setup As A Source Of Trust

Trust does not come from the fact that "AI is good". It comes from the fact that the system limits errors and makes responsibilities visible.

A robust agentic setup contains at least:

Element	Role
Versioned documentation	Gives agents and humans the same context.
Versioned AI instructions	Avoids personal and contradictory prompts.
Role-based agents	Developer, PM, Product Design, QA, review, security.
Reusable skills	Story writing, review, testing, documentation, audit.
Deterministic tools	Worktree, validation, local launch, report generation.
Properly sized CI/CD	Absorbs the human and agent implementation and validation flow.
PR environments	Enable functional validation before merge.
Code owners	Maintain technical accountability.
Regulation standup	Makes visible who takes what and with which delivery mode.
Shared tooling	Gives the whole team access to code, GitHub, agents and the same references.
Non-developer training	Enables PMs, POs[13], QA, CSMs and designers to test, read a PR and validate on a PR environment.

Two examples show why this setup is not cosmetic.

First example: GMS Runner. On this internal open-source project, an agent used without agentic configuration produced a fix, then committed and pushed it without validation. Repeating the rules in the conversation was not enough: the agent forgot them at the next iteration. After adding a short, versioned `AGENTS.md` describing TDD/ATDD and forbidding commits without explicit confirmation, the behavior changed: the agent wrote a test reproducing the bug, fixed it, validated it, noticed that the fix was incomplete, then iterated. Same model, same task, radically different result.

Second example: the QA agent. Developer agents often delivered correct code, but missed some specification elements. We therefore added an independent QA agent, functional rather than technical. It does not review the code: it compares the user story to the PR, tests the PR environment with Playwright, then produces a table of passed or failed acceptance criteria, with evidence.

The following table summarizes the principles to retain:

[13] **PO (Product Owner)**: Role responsible for clarifying the backlog, user stories and acceptance criteria in the delivery flow.

Topic	Decision
Instructions	Short, versioned, auditable. 600 lines is about 30 pages: too much for a human, too much for an agent.
GMS Runner example	Without `AGENTS.md`, the agent commits and pushes without validation. With TDD/ATDD and commit rules, same model, radically different behavior.
QA agent	Functional agent that compares story and PR, tests the PR environment, produces passed/failed ACs with evidence.
Non-developers	PMs/CSMs/QA can contribute, but never outside the flow: visible PR, functional validation by the person carrying the need, technical validation by the code owner.
Shared tooling	The tool may change; access to the repository, agents, PR environments and validation criteria must remain shared.
Training	PMs/POs/QA/CSMs must know how to read a PR, test a PR environment and comment acceptance criteria.

This separation is the heart of the model: democratize execution without diluting responsibility.

Target Organization

An efficient agentic organization is not necessarily larger. It is more explicit.

The real gain is not headcount reduction. It is the reduction of coordination cost: fewer handoffs, fewer internal queues, less implicit synchronization. The team makes decisions faster and sees problems earlier.

This does not mean skills disappear. The CSM, for example, did not exist in the initial team and was added because the customer voice needed to be closer to delivery. Conversely, QA is mostly transverse in this setup because the context is still prototyping. In a mature, critical or highly exposed product, that choice would probably be insufficient.

On my test project, the team composition evolved as follows:

Role	Before	After
Developers	3 FE[14] + 3 BE[15]	2.5 (full-stack)
QA	1 (100%)	0.2 (1 day/week, transverse expert)
DevOps[16]	1 (embedded)	0.3 (external support)
PM + PO	2 distinct people	1 (merged roles)
UX Designer (UXD[17])	1	0.5
CSM	- (did not exist)	1 (role added)
Total FTE (Full-Time Equivalent)	**~12**	**~4.5**

This is not an HR recipe. It is a prototyping context where the team was recomposed around delivery. The following table summarizes the organizational implications:

Point	Implication
2.5 dev FTE includes a junior	Explicit documentation + guardrails + PR review make onboarding safer.
Less coordination	Fewer handoffs, faster decisions, more visible problems.
Smaller team, lower resilience	Plan backup, transverse support and fast access to experts.
Reduced QA in this context	Acceptable in a prototype; not a recommendation for a mature or critical product.

[14] **FE (Front-End)**: Development of the user interface and client-side application layer.
[15] **BE (Back-End)**: Development of services, APIs, data and server-side processing.
[16] **DevOps**: Function that brings development and operations together: CI/CD, infrastructure, environments, observability, automation and delivery reliability.
[17] **UXD (User Experience Designer)**: Designer responsible for user experience, journeys, interfaces and usage consistency.

The right question is therefore not: “do we still need this role?” The right question is: “which skill do we need, how often, and with what level of risk?”

Need	Organizational decision
Needed 80-100% of the time	Integrate the skill into the team.
Occasional but critical need	Keep an external expert quickly accessible.
Low and recurring need	Train someone in the team, or establish a clear agreement with a transverse team able to absorb requests and organize itself.
Unknown need	Experiment, measure, then decide.

Roles partially become skills, but they do not disappear. QA, UX, DevOps, security, legal or architecture must be present depending on the frequency and criticality of the need. The QA agent helps validate earlier; it does not replace real quality accountability.

Concrete Starting Roadmap

Total cost of the experiment: 6 to 8 full-time FTE for the full duration (about 10 to 16 weeks depending on phases).

Requirement 0: Is The Organization Ready?

Microsoft calls this "Requirement 0". It is the prerequisite assessed by the external support team before deciding whether or not to invest 3-4 FTE to support the change.

The agentic experiment requires:

- **200% trust from management.** Not lukewarm agreement. A clear, explicit mandate that survives the first failure.
- **Actors who own their topic at 200%.** People who carry the initiative with conviction, not assigned executors.
- **Bulletproof motivation.** The experiment will go badly at times. The team must want to continue.
- **End-to-end ownership.** The team must be able to experiment as far as possible, solve its own problems or easily access experts to unblock itself.
- **No organizational bottlenecks.** If every technical decision requires going through a process, asking for authorization, waiting weeks for validation and prioritization, stop immediately. The organization is not ready.

If these conditions are not met, the investment will not produce a result. Requirement 0 is not a formality: it is an honest filter that avoids wasting time and trust.

Failing at setup is not a failure. It is learning. It does not mean stopping the change, but identifying blockers, understanding how to unblock the situation and moving forward.

The two sponsors play complementary roles:

- **The external sponsor (Microsoft/AWS)** challenges the team, accepts failure as normal, and may recommend postponing the rest of the project if the ground is not ready. Its role is to guide learning, not force the result.
- **The internal sponsor (Amadeus management)** escalates organizational problems and unblocks internal processes that slow the team down. Its role is to remove obstacles the team cannot solve alone.

Failures and blocking points encountered by the pilot team **must** be escalated to the sponsors. This feedback is not only used to unblock the project: it feeds the evolution of the organization as a whole. Every identified blocker is an opportunity to fix a process, an access issue, a rule or a tool that is probably slowing down other teams too.

Phase 0: Management Alignment

Recommended duration: 1 to 2 weeks.

Before launching a pilot, clarify the dominant objective:

Dominant objective	Associated trade-off
Learn	Accept failure and repetition.
Build autonomous agents	Temporarily slow delivery to invest in the setup.
Transform the method	Touch rituals, roles and responsibilities.
Prove Return on Investment (ROI)	Define metrics before the pilot.
Deliver faster	Advanced-phase objective, not the initial objective.

The management temptation is to launch the pilot with a promise of immediate productivity. That is the wrong entry point. If the team must prove ROI before understanding failures, stabilizing agents and adapting its flow, it will optimize the demonstration rather than learning.

Decision: do not start with "deliver faster". The first objective is to learn, stabilize the setup and align trade-offs. Speed comes later, as a consequence of a working system.

Phase 1: Prepare The Ground, Not The Whole Agentic System

Recommended duration: 2 to 4 weeks depending on existing maturity.
Human investment: ~4 FTE (1 tech lead/architect, 1 DevOps, 1 PM/PDA[18] for audit and repository preparation, 1 agentic expert to support adoption).
AI license cost: variable depending on tool (Copilot, Kiro, Claude Code, etc.), about EUR 20-50/user/month depending on the selected tool.

Do not fall into the opposite excess either: building the entire agentic system before experimenting. We do not yet know which agents, skills or tools will actually be useful. But we already know that some non-AI prerequisites are essential. An agent fails on a broken local environment for the same reasons as a newcomer. Slow or unstable CI/CD blocks agents as much as it blocks humans. Scattered documentation makes context fragile.

Decision: do not build the whole agentic system before trying. First prepare the non-AI ground: local environment, CI/CD, PR environments, documentation, tests, access.

Preparing the ground means:

[18] **PDA (Product Design Authority)**: Product/design framing role that helps align need, user experience, functional coherence and delivery trade-offs.

Element	Why it is essential
Reproducible local environment	Developers and agents must be able to test their changes under the same conditions. If the local environment does not work, the agent will fail for the same reasons as a newcomer.
Functional and stable CI/CD	You cannot build an acceleration system without stability. The pipeline must also be able to scale to absorb load peaks generated by humans and agents.
Change classification	CI/CD must distinguish a merge that affects the application from a merge that does not. A documentation change may publish a wiki; a dependency or code change may generate a new application version.
PR environments	They are a cornerstone of trust. They allow PMs, POs, QA, CSMs and developers to test before merge, therefore validate earlier and reduce correction cycles.
Centralized documentation	Inventorying product documentation is essential for future onboarding of engineers and agents. Hidden information kills quality.
Related repositories brought closer	If the experiment requires working across several repositories, bring them closer or put them in the same repository to smooth changes. Otherwise each repo adds coordination, divergence and friction.
Stable E2E tests	You do not need many at the beginning. They can come progressively. But a few reliable tests provide examples, a validation base and proof that the experiment can be controlled.
Tool access	This sounds obvious, but is often forgotten: if a human can do things the AI cannot do because it lacks access, the experiment will not work. GitHub, Figma, documentation and metrics must be accessible to the agentic system.
Aligned AI tooling	Tools may evolve, from VS Code to Kiro or something else, but the team must share the same references, agents, instructions and access. Otherwise each role develops its own shadow workflow.

The scope must be small, complete and controlled: frontend, backend, data, metrics, infrastructure, clear ownership. The agentic setup must emerge through adoption and proof, not through initial imposition.

Phase 2: Onboard The Agent

Recommended duration: 2 to 4 weeks, then continuous improvement.

An agent must be onboarded like a new team member. It needs rules, examples, access, limits and feedback. The difference is that a correction made to its

context can then benefit the whole team: humans, current agents and future agents.

Decision: onboard the agent like a newcomer. The objective is not yet to deliver faster; it is to understand what the agent needs to work correctly.

Agentic onboarding consists of progressively building:

Building block	Initial content
Project instructions	Minimal, versioned rules specific to the local context.
Basic agents	Developer, PM/story writer, review, QA.
Deterministic tools	Worktree creation, validation, local launch.
Access to information	GitHub, Figma, documentation, metrics.
Security guardrails	What the agent can do alone, what it must ask for, what it never does.
Agentic definition of done	Code, tests, validation, documentation, clear PR.
Feedback loops	Test, fail, correct, repeat.
Knowledge bases and examples	`ai-artifacts`[19] references, agents, skills and workflows that the team can pick from and adapt.

Also train non-developers: read a PR, test a PR environment, verify acceptance criteria, comment clearly. Without this, the setup remains a developer tool.

The phase must remain organic. The `ai-artifacts` bases serve as examples to pick from, not as an imposed framework.

Phase 3: Pilot With 1 To 2 Teams

Recommended duration: 6 to 8 weeks.
Recommended composition per pilot team: 2 developers, 1-2 functional profiles (QA/PDA/PM), a few support functions observing.

Choose a small team of early adopters who will iterate quickly to learn. The team must include an involved tech lead, an available PM/PO, and a QA or functional validator. Support functions (architecture, security, DevOps) must follow the pilot so they can later evangelize the rest of the organization.

External support: Microsoft and AWS offer to embed 2 to 4 engineers during the first weeks of adoption. This support cost is part of adoption pricing at cloud providers: they know it is the price of success.

The pilot must be real enough to produce useful learning, but limited enough to avoid putting the organization at risk.

Start with three types of work:

1. Small localized bugs.

[19] **ai-artifact**: Open-source framework that versions, composes and audits a project's agents, skills, tools and overlays. It enables reuse of upstream bases without lock-in.

2. Internal quality-of-life changes.
3. Documentation, tests and low-risk refactorings.

Avoid at the beginning: structural architecture, critical security, data migration, authentication, complex performance, irreversible changes.

For a critical product, integrate the QA focus from the pilot: regression, acceptance criteria, exploratory testing, critical paths, release quality.

Phase 4: Adapt Rituals

Recommended duration: during the pilot.

Adapt without renaming everything:

Ritual	Adaptation
Standup	Becomes a flow regulation and delegation decision point.
Refinement	Shorter, more focused on impact, risk and reversibility.
Demo	More frequent, often directly on a PR environment referenced in the Pull Request.
Planning	Lighter for small changes, continuous prioritization.
Retrospective	Analysis of agent failures, setup friction, missing instructions.

Phase 5: Measure And Decide Whether To Scale

Recommended duration: end of pilot, then monthly.

Measure only what guides the scale decision:

Dimension	Metric
Flow	Stories created, PRs opened, PRs merged, validations completed, releases.
Cycle time	Bug opened -> valid fix -> merge/prod.
CI/CD	Simultaneous builds, waiting time, flaky rate.
Validation	Time from story -> PR -> environment -> functional validation -> merge.
Adoption	Agent/skill/tool usage by role.
Satisfaction	Feedback from dev, PM, QA, CSM.
QA	Acceptance criteria validated/invalidated against the PR, regression, bugs found before/after merge.
Quality	Incidents, rollbacks, critical comments, breaking tests.

Scale only if the setup is stable, objectives are clear and metrics are readable. Otherwise, keep learning.

Investment And ROI

The full experiment represents about 3 months of concentrated investment:

- Phase 1 (1 month): ~4 FTE (tech lead/architect, DevOps, PM/PDA, support).
- Phases 2-3 (2 months): ~7 FTE (full pilot team + external support).
- **Total**: ~18 FTE-months (equivalent to 18 people mobilized for 1 month each).

This is not a net additional cost: most of this investment is existing team time redirected toward the experiment.

The return materializes through reduced coordination cost and the ability of the team to deliver with fewer people. On an initial team of 10-12 people, ROI is between 3 and 6 months after the end of the experiment. The reduced team can then accelerate on the current project, and the model can be replicated on other projects.

Risks And Mitigations

Risk	Mitigation
Initial setup failure	The team cannot structure, retrieve information or close technical gaps. If the target project is too large or too complex, do not force it: choose a smaller project, or invest significantly more time in the initial setup. It is better to postpone implementation on a large project than to approach it prematurely. Microsoft documented a success story on the .NET framework with hundreds of contributors; their approach is worth reading (see References).
Resistance to change	Do not try to convince everyone from the start. Begin with motivated and convinced early adopters. Skepticism softens through repeated small successful projects, not through theoretical arguments.
Skill loss	The risk exists. But code review builds programming skills: it is like reading a book and asking questions about it. There is no pure knowledge loss if teams work seriously. However, some reflexes will be forgotten, like when you create a script to make your life easier and forget what was inside. It is up to engineers to keep a minimum level of development hygiene.
Turnover and interruptions	On my test project, the AI expert left the company during the experiment. The new contacts had different skills and different perspectives; we learned differently and it was enriching. Long holidays, hackathons, constraints spanning several weeks: all of this impacts delivery, but learning happens and progress continues. An experiment is not a linear sprint. It absorbs disruption if the setup is versioned and documented.

Security And Compliance

Security comes up systematically in management discussions. It is legitimate, but it must not become an excuse not to move forward.

Points to address:

Topic	Approach
Data in LLMs	Major tools (GitHub Copilot, Claude Code, Kiro) do not retain submitted code for training in enterprise mode. Check contractual terms.
Intellectual property	Code generated by an agent follows the same rules as code written by a human: it goes through a PR, is reviewed, and belongs to the company.

Topic	Approach
Agent access perimeter	Define explicitly what the agent can do alone, what it must ask for, and what it never does. These guardrails are versioned in project instructions.
Regulatory compliance	Not different from a classic development tool. The agent does not deploy to production without human validation. The same controls apply.

Security is a configuration topic, not a fundamental blocker. Cloud providers have enterprise offerings with the required guarantees.

Continuation Criteria

Do not judge the experiment on the result of the first month. On my test project, if we had taken stock after one month, we would have stopped everything. The migration to Azure had been catastrophic, agents did not work well, everything was going wrong.

The feedback from our management and Microsoft was: failure is normal, it is fine. Now that we have something that works enough, we experiment and learn on top of it, we fix things progressively.

And we got there. Painfully, but it happened because the team drove it and top management provided unwavering support.

The real continuation criteria are not first-month productivity metrics. They are:

Criterion	Positive signal	Warning signal
Trust in the system	The team uses the setup naturally, agents produce useful work.	The team bypasses the setup, returns to old flows.
Visible monthly progress	Every month brings concrete improvements: fewer failures, better quality, a new use case unlocked.	Stagnation for several weeks without learning.
Management support	Management accepts failure as learning and maintains the mandate.	Management asks for immediate ROI and threatens to stop at the first problem.
Team engagement	Early adopters are enthusiastic and share their successes.	The team undergoes the experiment without believing in it.

Go/no-go is not binary. It is a monthly assessment of the trajectory, not of the instant result.

Practical Recommendations

1. **Start with the setup, not the promises.** Without a reliable environment, the agent fails and the team loses trust.
2. **Use a monorepo when possible.** CI/CD, infrastructure, documentation, CMS, UI and E2E tests in the same repo reduce synchronization and give agents a complete view.
3. **Prefer trunk-based development, small PRs and feature flags.** The objective is to reduce divergence time.
4. **Do not impose a central framework.** Governance must guide, document and support. Useful standards impose themselves because they prove their value.
5. **Give non-developers a role in the complete flow.** PMs, CSMs, QA and designers can contribute from story to validation if the setup is shared and technical validation remains clear.
6. **Train non-developers on the GitHub flow.** Reading a PR, testing a PR environment, verifying acceptance criteria and commenting a validation must become team skills.
7. **Treat failures as diagnosis.** A failing agent often reveals missing context, environment, tests or guardrails.
8. **Do not measure lines of code.** Measure end-to-end flow, validation, quality and adoption.

Cost Of Inaction

Not experimenting now is not a neutral position. It is a choice with consequences.

The cost of inaction cannot yet be measured precisely in euros. Precise ROI is still being evaluated on my test project, and scaling to other internal projects requires analyzing how each one works in order to adapt methods. That work will come.

What can already be measured is the learning delay:

- **Learning is cumulative and non-linear.** Organizations experimenting today accumulate months of know-how on agentic setup, failures and working patterns. This knowledge cannot be bought; it is built through iteration.
- **The gap widens every month.** Every month of practice improves agents, instructions, tools and flow. The longer we wait, the more delay there will be to catch up.
- **Attractiveness degrades.** The strongest engineers want to work with modern tools. An organization that does not experiment becomes less attractive.
- **Forced transformation is more expensive.** It is better to transform at your own pace, controlling choices, than to catch up under pressure.

The experiment costs ~18 FTE-months. Not experimenting costs a delay measured in quarters, not weeks.

Final Message

AI does not make an organization high-performing by magic.

It amplifies what already exists: scattered context, slow CI/CD, saturated review, misaligned objectives.

If the setup is solid, the effect is powerful: calmer developers, non-developers in the flow, earlier QA, field irritants handled, technical responsibility preserved.

The topic is therefore not adopting an AI tool.

The topic is building a delivery system where humans and agents can work together with trust.

References

1. AWS DevOps & Developer Productivity Blog, **AI-Driven Development Life Cycle: Reimagining Software Engineering**, Raja SP, 31 July 2025. https://aws.amazon.com/blogs/devops/ai-driven-development-life-cycle/
2. Microsoft .NET Blog, **Ten Months with Copilot Coding Agent in dotnet/runtime**, Stephen Toub, March 2026. Success story documenting large-scale deployment on a project with hundreds of contributors: 878 PRs, 67.9% merge rate. https://devblogs.microsoft.com/dotnet/ten-months-with-cca-in-dotnet-runtime/
3. Microsoft, **HVE Core**, repository used as an upstream source for RPI prompts in TSF. https://github.com/microsoft/hve-core
4. Monorepo Storefront used as a test project for the `ai-artifacts` framework, which versions, audits and composes reusable agents, skills, tools, overlays and knowledge bases. https://github.com/amadeus-nexwave/discovery-travelstorefront-monorepo